Exercise 1 — Classical generative models Generative neural networks for the sciences · Winter 2026/27

This notebook compares histogram, Gaussian, Gaussian mixture and kernel density models on noisy two moons, then Gaussian, GMM, KDE and the supplied density forest on 8 × 8 digits. All reported numbers and figures are produced by the code below. The two-dimensional estimators and both MMD kernels are implemented from scratch.

Reproduction. Python 3.10+, NumPy, SciPy, matplotlib, scikit-learn, pandas, Jupyter and threadpoolctl are required. Place `density_forest.py` beside this file. Run all cells in order. Experiments use fixed seeds, three repetitions for moons and two for digits; standard deviations describe these repetitions, not confidence intervals. This is a finite hyperparameter study rather than exhaustive tuning. Model selection uses validation data; the final test sets are reserved for the representative comparisons and visualizations.

Code cell [1]

```
import os, sys, time, warnings
from pathlib import Path
# Local optional dependency directory; ordinary installations do not need it.
if Path('.packages').is_dir():
    sys.path.insert(0, str(Path('.packages').resolve()))
os.environ['OMP_NUM_THREADS'] = '1'
os.environ['OPENBLAS_NUM_THREADS'] = '1'
import numpy as np
import pandas as pd
import scipy
from scipy.special import logsumexp
from scipy.spatial.distance import cdist
import matplotlib
import matplotlib.pyplot as plt
import sklearn
from sklearn.datasets import make_moons, load_digits
from sklearn.model_selection import train_test_split
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import KernelDensity
from sklearn.ensemble import RandomForestClassifier
from threadpoolctl import threadpool_limits
from IPython.display import display, Markdown
THREAD_LIMIT = threadpool_limits(limits=1)
plt.rcParams.update({'figure.dpi': 110, 'font.size': 10,

                     'axes.spines.top': False, 'axes.spines.right': False})
SEED = 20261105
print('Versions:', 'Python', sys.version.split()[0], 'NumPy', np.__version__,
      'SciPy', scipy.__version__, 'sklearn', sklearn.__version__,
      'matplotlib', matplotlib.__version__)
```

```
Versions: Python 3.10.11 NumPy 2.2.6 SciPy 1.15.3 sklearn 1.7.2 matplotlib 3.10.9
```

1. Two-dimensional estimators

All classes implement `fit(X)`, `sample(n_samples)` and `score_samples(X)`. In sklearn convention the last method returns **log density**, not density. The histogram assigns empirical probability mass to each bin and samples uniformly within the selected bin. Its fitted rectangular support is the training bounding box with a small padding; points outside it, and empty bins, have log density minus infinity. Increasing bin count reduces smoothing but increases variance.

The Gaussian uses the maximum-likelihood covariance (division by n). The GMM uses EM with log-space responsibilities, full covariance matrices, diagonal numerical regularization and several random initializations. Covariance regularization can slightly perturb exact EM monotonicity, so convergence is based on absolute

change in mean log likelihood. KDE is an equally weighted mixture centered at the training points with covariance h^2I. Sampling picks a training point and adds Gaussian noise.

Code cell [2]

```
def as_matrix(X):
    X = np.asarray(X, dtype=float)
    if X.ndim != 2 or len(X) == 0 or not np.isfinite(X).all():
        raise ValueError('Expected a nonempty finite n-by-d array')
    return X

def gaussian_logpdf(X, mean, cov):
    L = np.linalg.cholesky(cov)
    z = np.linalg.solve(L, (X - mean).T)
    return -.5 * (X.shape[1]*np.log(2*np.pi)
                  + 2*np.log(np.diag(L)).sum() + (z*z).sum(axis=0))

class Histogram2D:
    def __init__(self, bins=14, random_state=0):
        self.bins, self.random_state = bins, random_state

    def fit(self, X):
        X = as_matrix(X)
        if X.shape[1] != 2: raise ValueError('Histogram2D requires d=2')
        self.rng_ = np.random.default_rng(self.random_state)
        lo, hi = X.min(axis=0), X.max(axis=0)
        padding = np.maximum((hi-lo)*.025, 1e-6)
        self.edges_ = [np.linspace(lo[j]-padding[j], hi[j]+padding[j],
                                   self.bins+1) for j in range(2)]

        counts, _, _ = np.histogram2d(X[:, 0], X[:, 1], bins=self.edges_)
        self.mass_ = counts / len(X)
        self.area_ = np.outer(np.diff(self.edges_[0]), np.diff(self.edges_[1]))
        return self

    def sample(self, n_samples=1):
        flat = self.rng_.choice(self.mass_.size, n_samples,
                                p=self.mass_.ravel())
        ij = np.array(np.unravel_index(flat, self.mass_.shape)).T
        low = np.column_stack([self.edges_[j][ij[:, j]] for j in range(2)])
        high = np.column_stack([self.edges_[j][ij[:, j]+1] for j in range(2)])
        return self.rng_.uniform(low, high)

    def score_samples(self, X):
        X = as_matrix(X)
        ij = np.column_stack([np.searchsorted(self.edges_[j], X[:, j],
                                              side='right')-1 for j in range(2)])
        # The rightmost edge belongs to the last bin, as in np.histogram2d.
        for j in range(2):
            ij[X[:, j] == self.edges_[j][-1], j] = self.bins-1
        valid = ((ij >= 0) & (ij < self.bins)).all(axis=1)
        ans = np.full(len(X), -np.inf)
        idx = ij[valid]
        density = self.mass_[idx[:, 0], idx[:, 1]] / self.area_[idx[:, 0], idx[:, 1]]

        with np.errstate(divide='ignore'): ans[valid] = np.log(density)
        return ans

class SingleGaussian:
    def __init__(self, reg=1e-6, random_state=0):
        self.reg, self.random_state = reg, random_state

    def fit(self, X):
        X = as_matrix(X)
        self.mean_ = X.mean(axis=0)
        centered = X-self.mean_
        self.cov_ = centered.T @ centered / len(X) + self.reg*np.eye(X.shape[1])
        self.rng_ = np.random.default_rng(self.random_state)
        return self

    def sample(self, n_samples=1):
        return self.rng_.multivariate_normal(self.mean_, self.cov_, n_samples)

    def score_samples(self, X):
        return gaussian_logpdf(as_matrix(X), self.mean_, self.cov_)

class GMM:
```

```
    def __init__(self, n_components=6, reg=1e-6, max_iter=250, tol=1e-6,
                 n_init=3, random_state=0):

        self.n_components, self.reg = n_components, reg
        self.max_iter, self.tol, self.n_init = max_iter, tol, n_init
        self.random_state = random_state

    def fit(self, X):
        X = as_matrix(X)
        n, d = X.shape
        k = self.n_components
        if not 1 <= k <= n: raise ValueError('Require 1 <= components <= n')
        rng = np.random.default_rng(self.random_state)
        base = SingleGaussian(self.reg).fit(X)
        best = -np.inf
        for restart in range(self.n_init):
            means = X[rng.choice(n, k, replace=False)].copy()
            covs = np.repeat(base.cov_[None], k, axis=0)
            weights = np.ones(k)/k
            previous = -np.inf
            history = []
            for iteration in range(self.max_iter):
                logs = np.column_stack([gaussian_logpdf(X, means[j], covs[j])
                                        for j in range(k)]) + np.log(weights)
                norms = logsumexp(logs, axis=1)
                history.append(norms.mean())
                if abs(history[-1]-previous) < self.tol: break

                previous = history[-1]
                resp = np.exp(logs-norms[:, None])
                nk = resp.sum(axis=0)
                # Tiny-component safeguard; practically inactive for these runs.
                for j in np.flatnonzero(nk < 1e-8):
                    resp[:, j] = 1e-6
                nk = resp.sum(axis=0)
                weights = nk/nk.sum()
                means = resp.T @ X / nk[:, None]
                for j in range(k):
                    delta = X-means[j]
                    covs[j] = ((delta.T * resp[:, j]) @ delta / nk[j]
                               + self.reg*np.eye(d))
            final = np.mean(logsumexp(np.column_stack([
                gaussian_logpdf(X, means[j], covs[j]) for j in range(k)
            ]) + np.log(weights), axis=1))
            if final > best:
                best = final
                self.weights_, self.means_, self.covariances_ = (
                    weights.copy(), means.copy(), covs.copy())
                self.history_, self.n_iter_ = history, iteration+1
        self.rng_ = np.random.default_rng(self.random_state)
        return self

    def score_samples(self, X):
        X = as_matrix(X)
        logs = np.column_stack([gaussian_logpdf(X, m, c)
                  for m, c in zip(self.means_, self.covariances_)])
        return logsumexp(logs+np.log(self.weights_), axis=1)

    def sample(self, n_samples=1):
        choices = self.rng_.choice(self.n_components, n_samples, p=self.weights_)
        out = np.empty((n_samples, self.means_.shape[1]))
        for j in range(self.n_components):
            mask = choices == j
            out[mask] = self.rng_.multivariate_normal(
                self.means_[j], self.covariances_[j], mask.sum())
        return out

class GaussianKDE:
    def __init__(self, bandwidth=.15, random_state=0):
        self.bandwidth, self.random_state = bandwidth, random_state

    def fit(self, X):
        if self.bandwidth <= 0: raise ValueError('bandwidth must be positive')
        self.X_ = as_matrix(X).copy()
        self.rng_ = np.random.default_rng(self.random_state)
        return self

    def sample(self, n_samples=1):
        idx = self.rng_.integers(len(self.X_), size=n_samples)
```

```
        return self.X_[idx] + self.rng_.normal(
            scale=self.bandwidth, size=(n_samples, self.X_.shape[1]))

    def score_samples(self, X):
        X = as_matrix(X)
        h, d = self.bandwidth, self.X_.shape[1]
        result = []
        for chunk in np.array_split(X, max(1, int(np.ceil(len(X)/512)))):
            logs = -cdist(chunk, self.X_, 'sqeuclidean')/(2*h*h)
            result.append(logsumexp(logs, axis=1)-np.log(len(self.X_))
                          -d*np.log(h*np.sqrt(2*np.pi)))
        return np.concatenate(result)
```

2. Unbiased, vectorized MMD²

For independent samples X (n points) and Y (m points), we use the U-statistic

$MMD^2 = \sum_{i \neq j} k(x_i,x_j)/[n(n-1)] + \sum_{i \neq j} k(y_i,y_j)/[m(m-1)] - 2 \sum_{i,j} k(x_i,y_j)/(nm)$.

The squared exponential kernel is $\exp(-||x-y||^2/h^2)$, **without a factor 2**. The IMQ kernel is $(1 + ||x-y||^2/h^2)^{(-1/2)}$. Pairwise distances and kernels are vectorized. The estimate can be negative; no clipping is applied. Its quadratic memory requirement motivates evaluation with several hundred samples per run. Bandwidths are fixed from training distances, before any model comparison. Small bandwidths can make all pairwise similarities nearly zero; very large bandwidths make all similarities nearly one. Either limit hides model differences.

Code cell [3]

```
def kernel_from_d2(d2, h, kernel):
    if h <= 0: raise ValueError('h must be positive')
    if kernel == 'se': return np.exp(-d2/(h*h))
    if kernel == 'imq': return 1/np.sqrt(1+d2/(h*h))
    raise ValueError('Unknown kernel')

def mmd2(X, Y, h=1., kernel='se'):
    X, Y = as_matrix(X), as_matrix(Y)
    n, m = len(X), len(Y)
    if min(n, m) < 2: raise ValueError('At least two samples per set')
    Kxx = kernel_from_d2(cdist(X, X, 'sqeuclidean'), h, kernel)
    Kyy = kernel_from_d2(cdist(Y, Y, 'sqeuclidean'), h, kernel)
    Kxy = kernel_from_d2(cdist(X, Y, 'sqeuclidean'), h, kernel)
    return ((Kxx.sum()-np.trace(Kxx))/(n*(n-1))
            +(Kyy.sum()-np.trace(Kyy))/(m*(m-1))-2*Kxy.mean())

def median_distance(X):
    distances = cdist(X[:500], X[:500], 'euclidean')
    return np.median(distances[np.triu_indices(len(distances), 1)])

# Meaningful correctness checks, independent of the moon comparison.
check_X, _ = make_moons(n_samples=400, noise=.1, random_state=13)
normal = SingleGaussian(random_state=8).fit(check_X)
one = GMM(1, random_state=8).fit(check_X)

assert np.allclose(normal.mean_, one.means_[0], atol=1e-9)
assert np.allclose(normal.cov_, one.covariances_[0], atol=1e-9)
assert np.allclose(normal.score_samples(check_X), one.score_samples(check_X))
hist = Histogram2D(10).fit(check_X)
assert np.isclose(hist.mass_.sum(), 1)
centers = [(edges[:-1]+edges[1:])/2 for edges in hist.edges_]
centers_xy = np.array(np.meshgrid(*centers, indexing='ij')).reshape(2, -1).T
assert np.isclose(np.sum(np.exp(hist.score_samples(centers_xy))
                         *hist.area_.ravel()), 1)
assert np.isneginf(hist.score_samples(np.array([[100., 100.]])))[0]
for model in [hist, normal, one, GaussianKDE().fit(check_X)]:
    assert model.sample(17).shape == (17, 2)
    assert not np.isnan(model.score_samples(check_X)).any()
# KDE normalization over a sufficiently large grid.
axis = np.linspace(-3, 4, 350)
xx, yy = np.meshgrid(axis, axis)
grid = np.column_stack([xx.ravel(), yy.ravel()])
kde_mass = np.exp(GaussianKDE(.2).fit(check_X).score_samples(grid)).sum()
kde_mass *= (axis[1]-axis[0])**2
assert abs(kde_mass-1) < .01
A, B = np.array([[0.], [1.], [3.]]), np.array([[2.], [4.]])
```

```
    for kernel in ['se', 'imq']:
        f = lambda a,b: kernel_from_d2(np.sum((a-b)**2), 1.3, kernel)
        naive = (sum(f(a,b) for i,a in enumerate(A) for j,b in enumerate(A) if i != j)/6

                 +sum(f(a,b) for i,a in enumerate(B) for j,b in enumerate(B) if i != j)/2
                 -2*sum(f(a,b) for a in A for b in B)/6)
        assert np.allclose(mmd2(A, B, 1.3, kernel), naive)
        assert np.allclose(mmd2(A, B, 1.3, kernel), mmd2(B, A, 1.3, kernel))
    print('PASS: Gaussian == one-component GMM; normalized histogram and KDE;')
    print('sample shapes, finite/non-NaN scores, MMD naive formula and symmetry.')
    print('KDE numerical integral:', round(kde_mass, 6))
```

```
PASS: Gaussian == one-component GMM; normalized histogram and KDE;
sample shapes, finite/non-NaN scores, MMD naive formula and symmetry.
KDE numerical integral: 1.0
```

3. Moon experiments: sample size, smoothing and kernel bandwidth

Training sizes are 100, 500 and 1,500. Each experiment uses independently generated validation points (400) and generated points (400). We compare three histogram resolutions, four GMM component counts and three KDE bandwidths, plus the single Gaussian. Error bars show ±1 sample standard deviation across three seeds. The central MMD bandwidth is the median distance in an independent training-only reference sample. We also evaluate half and twice this bandwidth for both kernels.

Code cell [4]

```
MOON_CONFIGS = [('Histogram', b) for b in [6, 14, 35]]
MOON_CONFIGS += [('Gaussian', 1)] + [('GMM', k) for k in [1, 2, 6, 12]]
MOON_CONFIGS += [('KDE', h) for h in [.05, .15, .45]]
def moon_model(family, param, seed):
    if family == 'Histogram': return Histogram2D(int(param), seed)
    if family == 'Gaussian': return SingleGaussian(random_state=seed)
    if family == 'GMM': return GMM(int(param), random_state=seed)
    return GaussianKDE(param, seed)

ref, _ = make_moons(n_samples=500, noise=.1, random_state=SEED)
moon_h = median_distance(ref)
rows = []
start = time.time()
for n in [100, 500, 1500]:
    for rep in range(3):
        seed = SEED+100*rep+n
        X, _ = make_moons(n_samples=n, noise=.1, random_state=seed)
        val, _ = make_moons(n_samples=400, noise=.1, random_state=seed+1)
        for family, param in MOON_CONFIGS:
            model = moon_model(family, param, seed).fit(X)
            generated = model.sample(400)
            for kernel in ['se', 'imq']:
                for factor in [.5, 1., 2.]:
                    rows.append(dict(n=n, rep=rep, family=family, param=param,

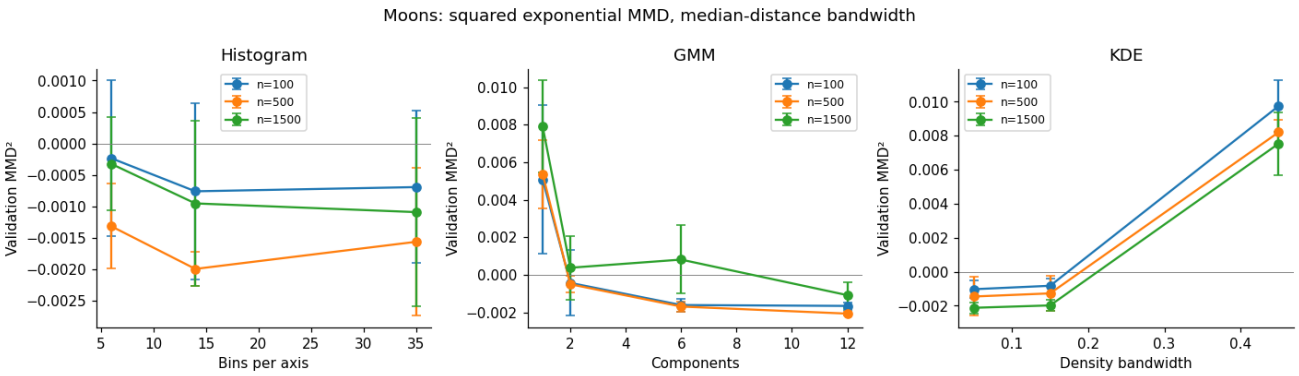
                        kernel=kernel, factor=factor,
                        mmd=mmd2(val, generated, moon_h*factor, kernel)))
moon_results = pd.DataFrame(rows)
moon_results.to_csv('moon_results.csv', index=False)
print(f'Moon sweep: {time.time()-start:.1f} s; reference bandwidth = {moon_h:.3f}')
central = moon_results.query("kernel == 'se' and factor == 1")
fig, axes = plt.subplots(1, 3, figsize=(13, 3.8))
for ax, family in zip(axes, ['Histogram', 'GMM', 'KDE']):
    subset = central[central.family == family]
    for n, group in subset.groupby('n'):
        stats = group.groupby('param').mmd.agg(['mean', 'std'])
        ax.errorbar(stats.index, stats['mean'], yerr=stats['std'], marker='o',
                    capsize=3, label=f'n={n}')
    ax.set(xlabel={'Histogram':'Bins per axis', 'GMM':'Components',
                   'KDE':'Density bandwidth'}[family], ylabel='Validation MMD²',
           title=family)
    ax.axhline(0, color='grey', lw=.6)
    ax.legend(fontsize=8)
fig.suptitle('Moons: squared exponential MMD, median-distance bandwidth')
fig.tight_layout(); plt.show()

summary = central.groupby(['n', 'family', 'param']).mmd.agg(['mean','std'])
```

```
print(summary.round(6).to_string())

Moon sweep: 16.0 s; reference bandwidth = 1.172
                          mean       std
n    family    param
100  GMM       1.00   0.005084  0.003945
               2.00  -0.000429  0.001759
               6.00  -0.001609  0.000344
               12.00 -0.001656  0.000196
     Gaussian  1.00   0.007766  0.002318
     Histogram 6.00  -0.000235  0.001235
               14.00 -0.000760  0.001399
               35.00 -0.000693  0.001211
     KDE       0.05  -0.001027  0.000538
               0.15  -0.000825  0.000419
               0.45   0.009748  0.001500
500  GMM       1.00   0.005376  0.001816
               2.00  -0.000502  0.000450
               6.00  -0.001689  0.000265
               12.00 -0.002066  0.000109
     Gaussian  1.00   0.005352  0.003517
     Histogram 6.00  -0.001315  0.000672
               14.00 -0.001993  0.000270
               35.00 -0.001563  0.001177
     KDE       0.05  -0.001456  0.001140
               0.15  -0.001272  0.001050

               0.45   0.008206  0.000743
1500 GMM       1.00   0.007901  0.002451
               2.00   0.000378  0.001689
               6.00   0.000811  0.001818
               12.00 -0.001082  0.000667
     Gaussian  1.00   0.006993  0.000641
     Histogram 6.00  -0.000324  0.000743
               14.00 -0.000953  0.001312
               35.00 -0.001091  0.001503
     KDE       0.05  -0.002119  0.000324
               0.15  -0.001980  0.000307
               0.45   0.007536  0.001854
```



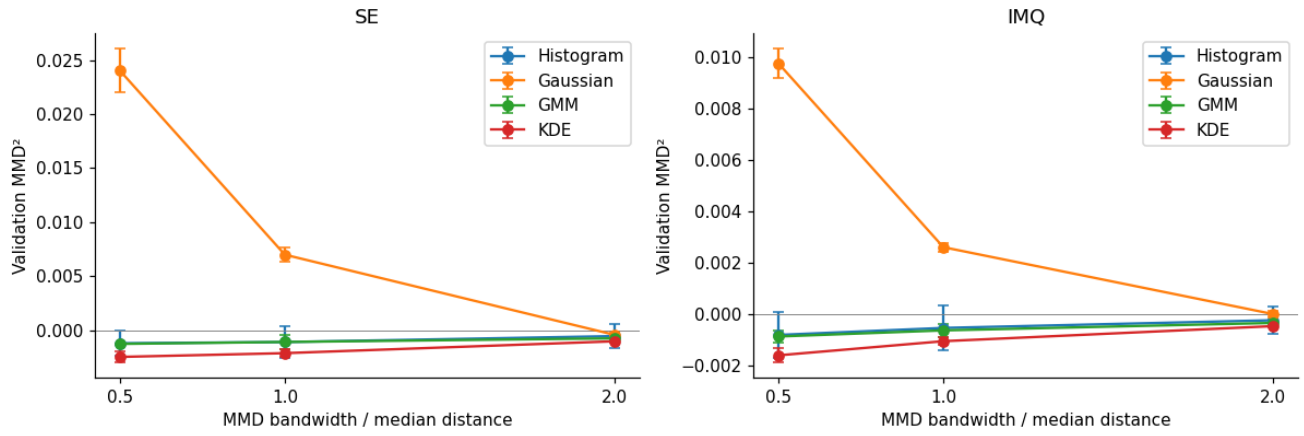
Code cell [5]

```
# Select per-family settings on validation averages at the largest n only.
best_moon = {}
for family in ['Histogram', 'Gaussian', 'GMM', 'KDE']:
    g = central[(central.n == 1500) & (central.family == family)]
    best_moon[family] = g.groupby('param').mmd.mean().idxmin()
fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
for ax, kernel in zip(axes, ['se', 'imq']):
    for family, param in best_moon.items():
        g = moon_results[(moon_results.n == 1500)
                         & (moon_results.family == family)
                         & (moon_results.param == param)
                         & (moon_results.kernel == kernel)]
        stats = g.groupby('factor').mmd.agg(['mean','std'])
        ax.errorbar(stats.index, stats['mean'], yerr=stats['std'], marker='o',
                    capsize=3, label=family)
    ax.set(xlabel='MMD bandwidth / median distance', ylabel='Validation MMD²',
           title=kernel.upper())
    ax.set_xticks([.5, 1, 2]); ax.axhline(0, color='grey', lw=.6); ax.legend()
fig.tight_layout(); plt.show()
print('Validation-selected settings:', best_moon)

# Independent test: never used to select settings.
train_final, _ = make_moons(n_samples=1500, noise=.1, random_state=SEED+9000)
moon_test, _ = make_moons(n_samples=1000, noise=.1, random_state=SEED+9001)

test_rows = []
for family, param in best_moon.items():
    model = moon_model(family, param, SEED+9002).fit(train_final)
    generated = model.sample(1000)
    test_rows.append(dict(model=family, setting=param,
        SE=mmd2(moon_test, generated, moon_h, 'se'),
        IMQ=mmd2(moon_test, generated, moon_h, 'imq')))
moon_test_scores = pd.DataFrame(test_rows)
print('Independent test (1,000 points per distribution):')
print(moon_test_scores.round(6).to_string(index=False))
```

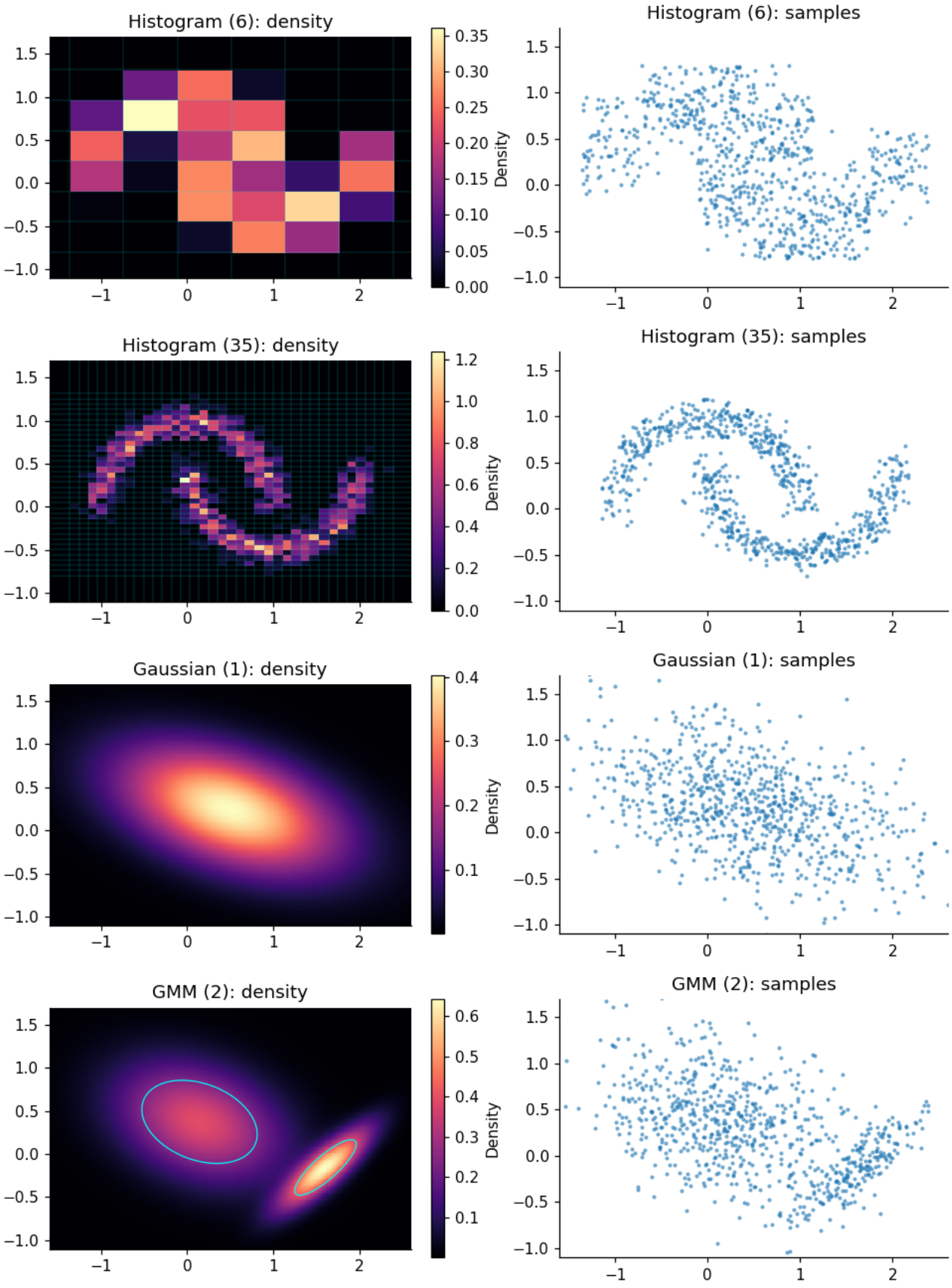
```
Validation-selected settings: {'Histogram': np.float64(35.0), 'Gaussian': np.float64(1.0), 'GMM':
    np.float64(12.0), 'KDE': np.float64(0.05)}
Independent test (1,000 points per distribution):
    model  setting        SE       IMQ
Histogram    35.00 -0.000985 -0.000507
 Gaussian     1.00  0.007211  0.002870
      GMM    12.00 -0.000171 -0.000023
      KDE     0.05 -0.000947 -0.000452
```

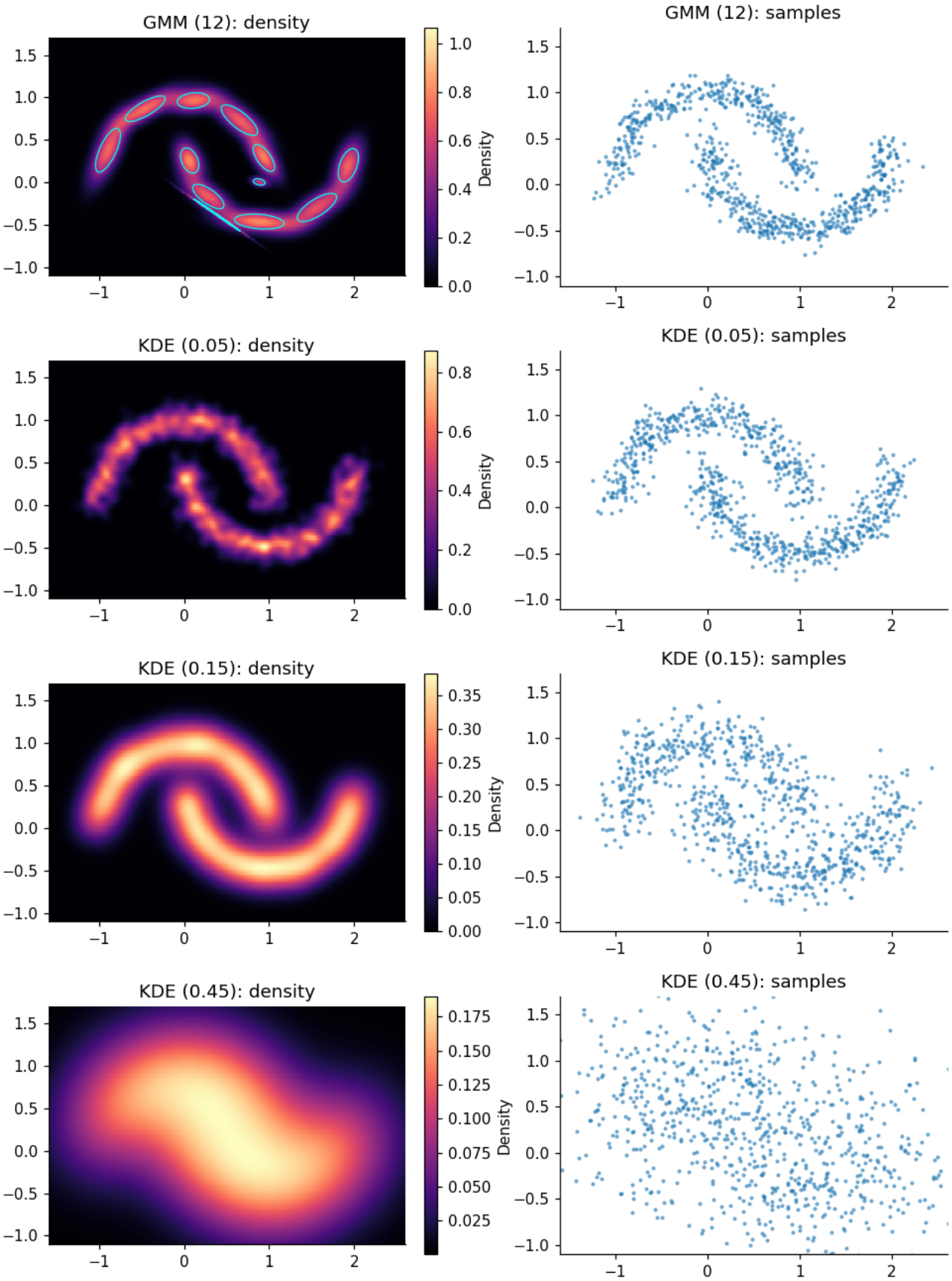


Code cell [6]

```
# Two outputs for every representative: numerical density and generated data.
representatives = [('Histogram', 6), ('Histogram', 35), ('Gaussian', 1),
                   ('GMM', 2), ('GMM', 12), ('KDE', .05), ('KDE', .15), ('KDE', .45)]
x_axis, y_axis = np.linspace(-1.6, 2.6, 180), np.linspace(-1.1, 1.7, 140)
gx, gy = np.meshgrid(x_axis, y_axis)
grid = np.column_stack([gx.ravel(), gy.ravel()])
for block in [representatives[:4], representatives[4:]]:
    fig, axes = plt.subplots(len(block), 2, figsize=(9, 3*len(block)))
    for row, (family, param) in enumerate(block):
        model = moon_model(family, param, SEED+300).fit(train_final)
        density = np.exp(model.score_samples(grid)).reshape(gx.shape)
        im = axes[row, 0].pcolormesh(gx, gy, density, shading='auto', cmap='magma')
        fig.colorbar(im, ax=axes[row, 0], label='Density', fraction=.04)
        generated = model.sample(800)
        axes[row, 1].scatter(*generated.T, s=3, alpha=.5)
        if family == 'Histogram':
            for edge in model.edges_[0]: axes[row, 0].axvline(edge, lw=.25, color='cyan', alpha=.5)
            for edge in model.edges_[1]: axes[row, 0].axhline(edge, lw=.25, color='cyan', alpha=.5)
        if family == 'GMM':
            from matplotlib.patches import Ellipse
            for mean, cov in zip(model.means_, model.covariances_):
                vals, vecs = np.linalg.eigh(cov)
                angle = np.degrees(np.arctan2(vecs[1, -1], vecs[0, -1]))
                axes[row, 0].add_patch(Ellipse(mean, 2*np.sqrt(vals[-1]),

                    2*np.sqrt(vals[0]), angle=angle, fill=False, ec='cyan', lw=.8))
        for j, ax in enumerate(axes[row]):
            ax.axis('equal')
            ax.set_aspect('equal', adjustable='box')
            ax.set(xlim=(-1.6, 2.6), ylim=(-1.1, 1.7),
                   title=f'{family} ({param:g}): '+['density', 'samples'][j])
    fig.tight_layout(); plt.show()
```


Histogram (6): density
Histogram (6): samples
Density
Histogram (35): density
Histogram (35): samples
Density
Gaussian (1): density
Gaussian (1): samples
Density
GMM (2): density
GMM (2): samples
Density


GMM (12): density
GMM (12): samples
KDE (0.05): density
KDE (0.05): samples
KDE (0.15): density
KDE (0.15): samples
KDE (0.45): density
KDE (0.45): samples
Density

Code cell [7]

```
ranking = moon_test_scores.sort_values('SE')
display(Markdown('### Moon observations\n'
    f"On the independent test set, the smallest SE MMD² is **{ranking.iloc[0]['model']}** "
    f"({ranking.iloc[0]['SE']:.5f}); the single Gaussian has "
    f"{moon_test_scores.set_index('model').loc['Gaussian','SE']:.5f}. "
    'These are finite-sample estimates, so small differences should not be overinterpreted.\n\n'
    'A single Gaussian fills the space between the two curved modes and cannot express their shape. '
    'Mixtures use local ellipses to follow the arcs; few components underfit, while extra components '
    'cost more and can fit sampling noise. Coarse histogram bins blur the arcs; fine bins create '
    'empty regions and block boundaries, especially with little training data. KDE with a narrow '
    'bandwidth preserves local structure but can retain isolated sample bumps; a wide bandwidth '
    'merges the moons. Additional data mainly helps the more flexible settings. The sweep figures '
    'show the actual size of these effects, rather than assuming every curve is monotonic.\n\n'
    'SE and IMQ put different emphasis on distances; their absolute MMD values are not comparable. '
    'Bandwidth sensitivity means a small score at one h alone is insufficient. Negative estimates '
    'are consistent with the unbiased estimator and do not imply negative population MMD².'))
```

Moon observations On the independent test set, the smallest SE MMD² is Histogram (-0.00099); the single Gaussian has 0.00721. These are finite-sample estimates, so small differences should not be overinterpreted.

A single Gaussian fills the space between the two curved modes and cannot express their shape. Mixtures use local ellipses to follow the arcs; few components underfit, while extra components cost more and can fit sampling noise. Coarse histogram bins blur the arcs; fine bins create empty regions and block boundaries, especially with little training data. KDE with a narrow bandwidth preserves local structure but can retain isolated sample bumps; a wide bandwidth merges the moons. Additional data mainly helps the more flexible settings. The sweep figures show the actual size of these effects, rather than assuming every curve is monotonic.

SE and IMQ put different emphasis on distances; their absolute MMD values are not comparable. Bandwidth sensitivity means a small score at one h alone is insufficient. Negative estimates are consistent with the unbiased estimator and do not imply negative population MMD².

4. Digits in 64 dimensions

Pixels are divided by 16, putting the original intensity scale in [0,1]. We split the 1,797 examples into 1,347 development and 450 **test** examples, stratified by digit. The development set is split into 1,200 training and 147 validation examples. Training sizes are 300, 700 and 1,200; training subsets are nested within each seed. No label is supplied to any density model. A classifier uses training labels only.

Models: sklearn full-covariance Gaussian (GMM with K=1), diagonal GMM with K in {5,15,30}, Gaussian KDE with h in {0.05,0.15,0.4}, and the supplied density forest with 2 trees and minimum leaf sizes 80 and 160. The large leaves stabilize 64-dimensional covariance estimates. Diagonal GMM covariance limits the parameter count; this is a stated modelling choice, not an implementation requirement.

Provided forest. The class is downloaded from the assignment link and included unchanged in the archive. Its leaves fit Gaussians rather than uniform bin densities, as explained in its docstring. For reproducibility we fit the same supplied trees sequentially, with explicit seeds, avoiding its unseeded multiprocessing wrapper. For this model only we add Gaussian training jitter of standard deviation 0.001 in normalized pixel units to avoid singular determinants from constant background pixels. No PCA or class conditioning is used. The NumPy 2 compatibility alias `np.infty = np.inf` is set below; the source file itself is preserved.

MMD uses original **unclipped** generated vectors for every model. Clipping to [0,1] is used only for displaying images and classifier evaluation, since the classifier is trained on valid pixel intensities. The out-of-range fraction is reported so this display choice cannot hide support violations.

Code cell [8]

```
from density_forest import DensityForest
np.infty = np.inf  # Compatibility for the supplied pre-NumPy-2 source.
digits = load_digits()
X_all, y_all = digits.data/16., digits.target
X_dev, X_test, y_dev, y_test = train_test_split(
    X_all, y_all, test_size=450, stratify=y_all, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, train_size=1200, stratify=y_dev, random_state=SEED+1)
digit_h = median_distance(X_train)
print('Train / validation / test:', len(X_train), len(X_val), len(X_test))
print('Median training distance:', round(digit_h, 4))

def fit_digit_model(family, param, X, seed):
    if family == 'Gaussian':
        return GaussianMixture(n_components=1, covariance_type='full',
                    reg_covar=1e-4, random_state=seed).fit(X)
    if family == 'GMM':
        return GaussianMixture(n_components=int(param), covariance_type='diag',
                    reg_covar=1e-4, n_init=2, max_iter=300,
                    random_state=seed).fit(X)
    if family == 'KDE':
        return KernelDensity(bandwidth=param).fit(X)
    model = DensityForest(n_trees=2, n_min=int(param))
    jittered = X + np.random.default_rng(seed).normal(0, .001, X.shape)

    for j, tree in enumerate(model.trees):
        np.random.seed(seed+j)
        model.trees[j] = model.fit_tree(tree, jittered)
    return model

def digit_sample(model, family, n, seed):
    if family in ['Gaussian', 'GMM']:
        model.random_state = seed
        return model.sample(n)[0]
    if family == 'KDE': return model.sample(n, random_state=seed)
    np.random.seed(seed)
    return model.sample(n)

DIGIT_CONFIGS = [('Gaussian', 1)] + [('GMM', k) for k in [5, 15, 30]]
DIGIT_CONFIGS += [('KDE', h) for h in [.05, .15, .4]]
DIGIT_CONFIGS += [('Forest', nmin) for nmin in [80, 160]]
digit_rows, fitted_digits = [], {}
start = time.time()
for rep in range(2):
    order = np.random.default_rng(SEED+rep).permutation(len(X_train))
    for n in [300, 700, 1200]:
        X = X_train[order[:n]]
        for family, param in DIGIT_CONFIGS:
            seed = SEED+rep*100+n

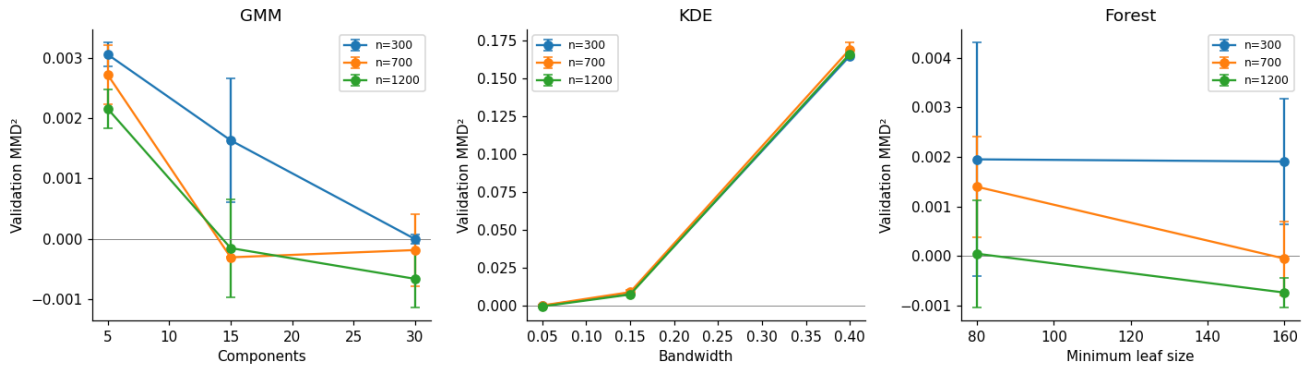
            tick = time.time()
            model = fit_digit_model(family, param, X, seed)
            generated = digit_sample(model, family, 400, seed+1)
            assert generated.shape == (400, 64) and np.isfinite(generated).all()
            for kernel in ['se', 'imq']:
                for factor in [.5, 1., 2.]:
                    digit_rows.append(dict(n=n, rep=rep, family=family, param=param,
                        kernel=kernel, factor=factor,
                        mmd=mmd2(X_val, generated, digit_h*factor, kernel)))
            if n == 1200 and rep == 0:
                fitted_digits[(family, param)] = model
        print(f'Digits complete: repetition={rep+1}, n={n}, elapsed={time.time()-start:.1f}s')
digit_results = pd.DataFrame(digit_rows)
digit_results.to_csv('digit_results.csv', index=False)
digit_central = digit_results.query("kernel == 'se' and factor == 1")
fig, axes = plt.subplots(1, 3, figsize=(13, 3.8))
for ax, family in zip(axes, ['GMM', 'KDE', 'Forest']):
    for n, g in digit_central[digit_central.family == family].groupby('n'):
        stats = g.groupby('param').mmd.agg(['mean','std'])
        ax.errorbar(stats.index, stats['mean'], yerr=stats['std'], marker='o',
                    capsize=3, label=f'n={n}')
    ax.set(title=family, ylabel='Validation MMD²',
           xlabel={'GMM':'Components', 'KDE':'Bandwidth', 'Forest':'Minimum leaf size'}[family])
    ax.axhline(0, color='grey', lw=.6); ax.legend(fontsize=8)

fig.tight_layout(); plt.show()
```

```
  print(digit_central.groupby(['n','family','param']).mmd.agg(['mean','std'])
        .round(6).to_string())

Train / validation / test: 1200 147 450
Median training distance: 3.0727
Digits complete: repetition=1, n=300, elapsed=1.5s
Digits complete: repetition=1, n=700, elapsed=17.9s
Digits complete: repetition=1, n=1200, elapsed=96.3s
Digits complete: repetition=2, n=300, elapsed=97.7s
Digits complete: repetition=2, n=700, elapsed=114.0s
Digits complete: repetition=2, n=1200, elapsed=191.1s
                          mean       std
n    family   param
300  Forest   80.00   0.001950  0.002356
              160.00  0.001906  0.001276
     GMM      5.00    0.003065  0.000194
              15.00   0.001636  0.001028
              30.00  -0.000007  0.000084
     Gaussian 1.00    0.000636  0.000519
     KDE      0.05   -0.000327  0.000571
              0.15    0.007547  0.000544
              0.40    0.164587  0.000758
700  Forest   80.00   0.001396  0.001015
              160.00 -0.000056  0.000748
     GMM      5.00    0.002729  0.000487
              15.00  -0.000310  0.000011
              30.00  -0.000189  0.000596

     Gaussian 1.00    0.000422  0.000099
     KDE      0.05   -0.000131  0.000385
              0.15    0.008676  0.001447
              0.40    0.169008  0.004283
1200 Forest   80.00   0.000042  0.001084
              160.00 -0.000743  0.000299
     GMM      5.00    0.002163  0.000326
              15.00  -0.000156  0.000816
              30.00  -0.000668  0.000474
     Gaussian 1.00   -0.000333  0.000260
     KDE      0.05   -0.000757  0.000630
              0.15    0.007088  0.000596
              0.40    0.165953  0.001214
```



Code cell [9]

```
best_digits = {}
for family in ['Gaussian', 'GMM', 'KDE', 'Forest']:
    g = digit_central[(digit_central.n == 1200) & (digit_central.family == family)]
    best_digits[family] = g.groupby('param').mmd.mean().idxmin()
print('Validation-selected settings:', best_digits)

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
for ax, kernel in zip(axes, ['se', 'imq']):
    for family, param in best_digits.items():
        g = digit_results[(digit_results.n == 1200)
            & (digit_results.family == family) & (digit_results.param == param)
            & (digit_results.kernel == kernel)]
        stats = g.groupby('factor').mmd.agg(['mean','std'])
        ax.errorbar(stats.index, stats['mean'], yerr=stats['std'], marker='o',
                    capsize=3, label=family)
    ax.set(title=kernel.upper(), xlabel='MMD bandwidth / median distance',
           ylabel='Validation MMD²')
    ax.set_xticks([.5, 1, 2]); ax.legend()
fig.tight_layout(); plt.show()

rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=1,
                            random_state=SEED, n_jobs=1).fit(X_train, y_train)
print(f'Classifier accuracy on untouched test digits: {rf.score(X_test,y_test):.3%}')
print('Actual test digit proportions:', np.round(np.bincount(y_test)/len(y_test), 3))

digit_final, generated_digits, proportions = [], {}, {}
for family, param in best_digits.items():
    model = fitted_digits[(family, param)]
    generated = digit_sample(model, family, 1000, SEED+8800)
    generated_digits[family] = generated
    probabilities = rf.predict_proba(np.clip(generated, 0, 1))
    labels, confidence = probabilities.argmax(axis=1), probabilities.max(axis=1)
    recognizable = confidence >= .6
    all_counts = np.bincount(labels, minlength=10)/len(labels)
    recog_counts = np.bincount(labels[recognizable], minlength=10)
    conditional = recog_counts/max(1, recognizable.sum())
    proportions[family] = (all_counts, conditional)
    digit_final.append(dict(model=family, setting=param,
        SE=mmd2(X_test, generated, digit_h, 'se'),
        IMQ=mmd2(X_test, generated, digit_h, 'imq'),
        out_of_range=np.mean((generated < 0) | (generated > 1)),
        confident_fraction=recognizable.mean(), mean_confidence=confidence.mean(),
        TV_all=.5*np.abs(all_counts-.1).sum(),
        TV_confident=(.5*np.abs(conditional-.1).sum()
                      if recognizable.any() else np.nan)))
digit_test_scores = pd.DataFrame(digit_final)
print('Final test and classifier diagnostics:')
print(digit_test_scores.round(5).to_string(index=False))

digit_test_scores.to_csv('digit_test_scores.csv', index=False)

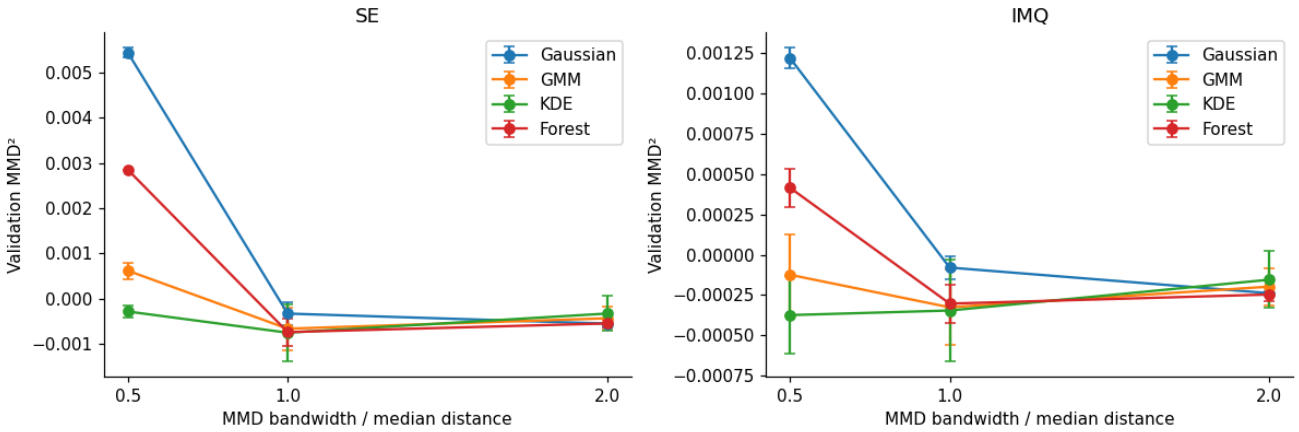
# Unfiltered random draws: do not cherry-pick recognizable digits.
show_configs = [('Gaussian',1), ('GMM',5), ('GMM',best_digits['GMM']),
                ('KDE',.4), ('KDE',best_digits['KDE']), ('Forest',best_digits['Forest'])]
for family, param in show_configs:
    samples = digit_sample(fitted_digits[(family,param)], family, 24, SEED+99)
    fig, axes = plt.subplots(3, 8, figsize=(9, 3.6))
    for ax, sample in zip(axes.ravel(), samples):
        ax.imshow(np.clip(sample.reshape(8,8),0,1), cmap='gray', vmin=0, vmax=1)
        ax.axis('off')
    fig.suptitle(f'{family}, parameter={param:g}: 24 unfiltered random samples')
    fig.tight_layout(); plt.show()

fig, axes = plt.subplots(2, 2, figsize=(11, 7))
for ax, (family, (all_p, conditional)) in zip(axes.ravel(), proportions.items()):
    ax.bar(np.arange(10)-.18, all_p, width=.36, label='All draws')
    ax.bar(np.arange(10)+.18, conditional, width=.36, label='Confidence >= 0.6')
    ax.axhline(.1, ls='--', color='black', label='Uniform target')
    ax.set(title=family, xlabel='Classifier digit', ylabel='Proportion',
           xticks=np.arange(10), ylim=(0, max(.22, 1.1*max(all_p.max(),conditional.max()))))
    ax.legend(fontsize=8)
fig.tight_layout(); plt.show()
print('Confidence threshold sensitivity (fraction retained):')

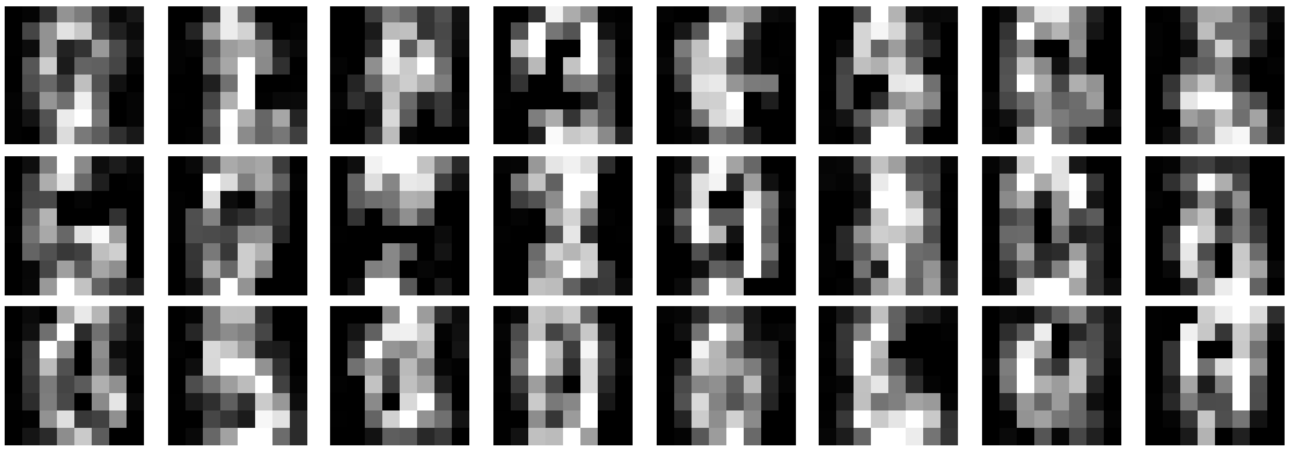
for family, generated in generated_digits.items():
    confidence = rf.predict_proba(np.clip(generated,0,1)).max(axis=1)
```

```
      print(family, {t: round(float(np.mean(confidence >= t)),3) for t in [.4,.6,.8]})

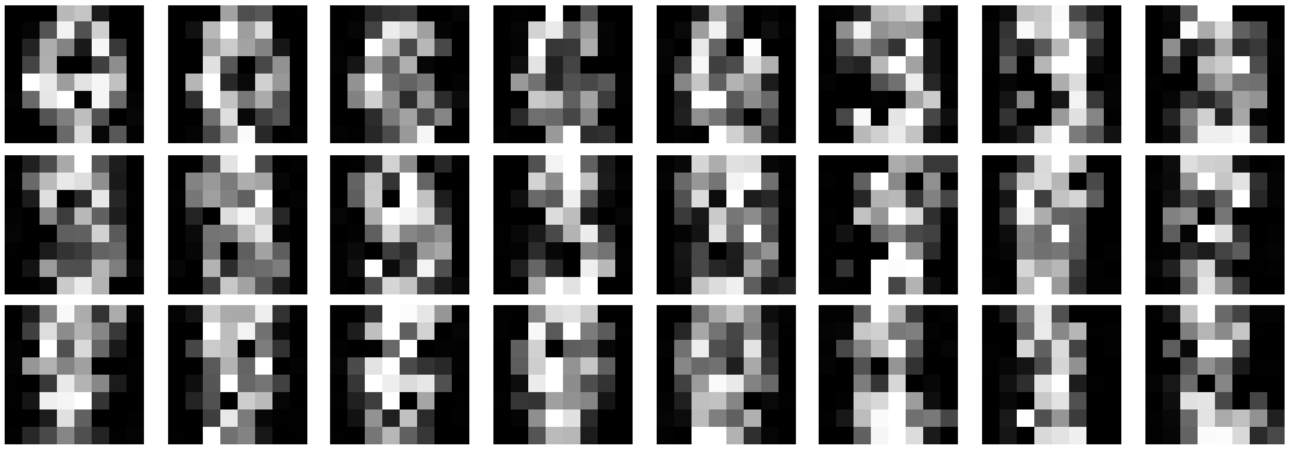
Validation-selected settings: {'Gaussian': np.float64(1.0), 'GMM': np.float64(30.0), 'KDE':
    np.float64(0.05), 'Forest': np.float64(160.0)}
Classifier accuracy on untouched test digits: 97.111%
Actual test digit proportions: [0.1   0.102 0.098 0.102 0.1   0.102 0.1   0.1   0.096 0.1  ]
Final test and classifier diagnostics:
   model  setting       SE      IMQ  out_of_range  confident_fraction  mean_confidence  TV_all
    TV_confident
Gaussian     1.00  0.00121  0.00064       0.27947               0.036          0.33459   0.162
    0.29444
     GMM    30.00  0.00019  0.00006       0.28567               0.699          0.68573   0.028
    0.10114
     KDE     0.05 -0.00001 -0.00001       0.29786               0.873          0.75182   0.027
    0.04296
  Forest   160.00  0.00046  0.00031       0.28769               0.160          0.42732   0.122
    0.24375
Confidence threshold sensitivity (fraction retained):
Gaussian {0.4: 0.246, 0.6: 0.036, 0.8: 0.001}
GMM {0.4: 0.934, 0.6: 0.699, 0.8: 0.298}
KDE {0.4: 0.995, 0.6: 0.873, 0.8: 0.399}
Forest {0.4: 0.482, 0.6: 0.16, 0.8: 0.046}
```



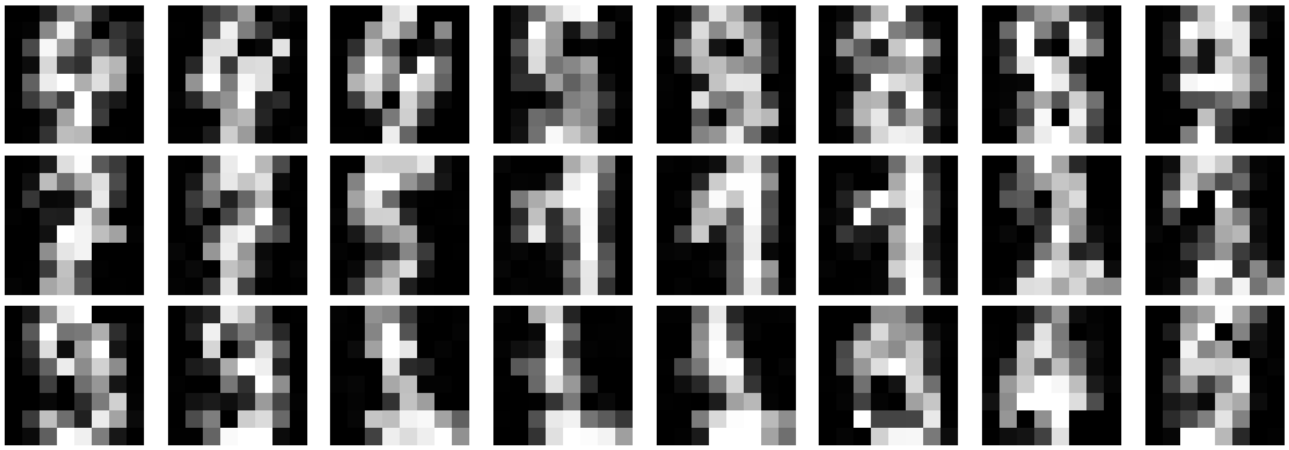
Gaussian, parameter=1: 24 unfiltered random samples



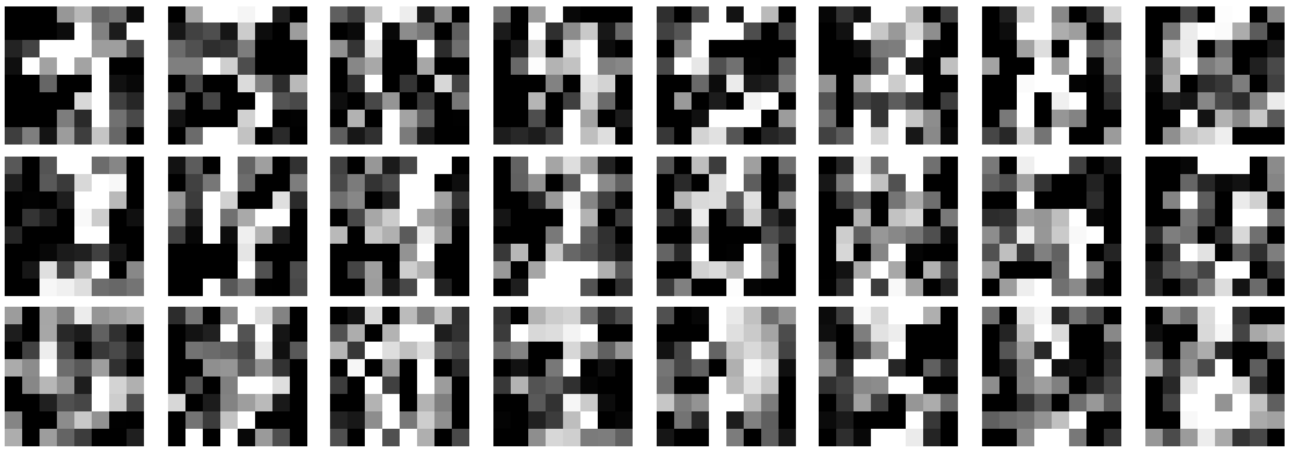
GMM, parameter=5: 24 unfiltered random samples



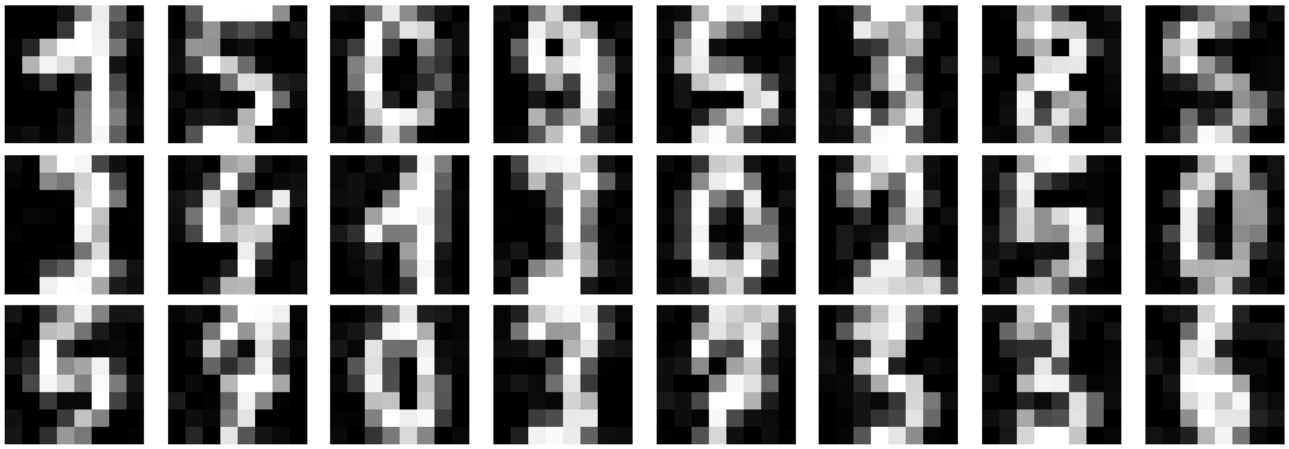
GMM, parameter=30: 24 unfiltered random samples



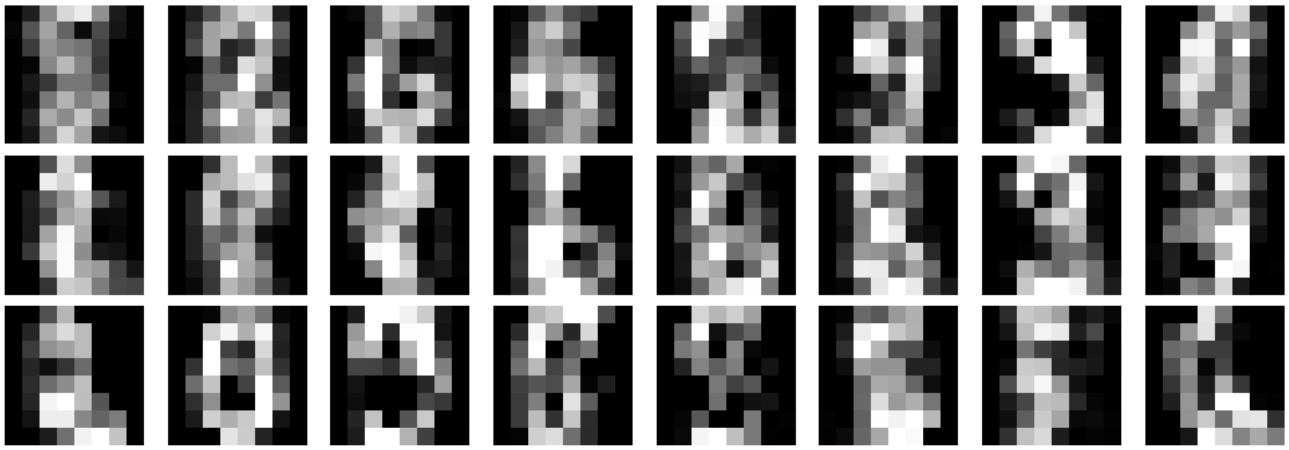
KDE, parameter=0.4: 24 unfiltered random samples

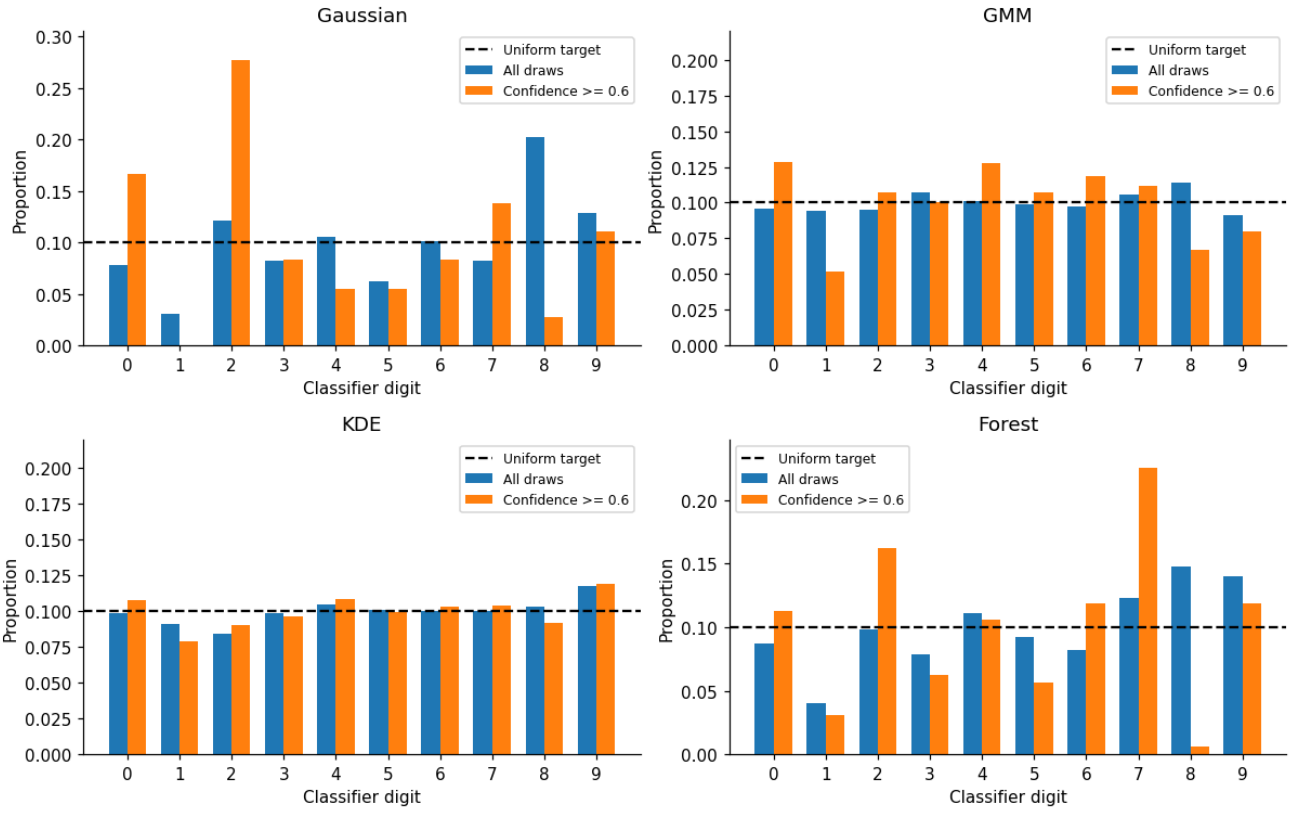


KDE, parameter=0.05: 24 unfiltered random samples



Forest, parameter=160: 24 unfiltered random samples





Code cell [10]

```
rank = digit_test_scores.sort_values('SE')
lines = ['### Digit observations',
    f"The independent-test SE ranking is: **{' → '.join(rank.model)}** "
    '(lower MMD² first). The table reports IMQ separately; agreement between '
    'kernels is stronger evidence than one score alone.',
    'A full single Gaussian models global covariance but averages distinct digit modes. '
    'A mixture separates some modes; the diagonal restriction loses within-component '
    'pixel correlations, and more components use more parameters. KDE preserves local '
    'training images at a small bandwidth, but this does not by itself establish '
    'novel sample generation. Large KDE bandwidths inject visible pixel noise. '
    'The supplied forest partitions the data and fits local full Gaussians; its '
    'minimum leaf size trades local specialization against covariance stability.',
    'The displayed samples are unfiltered random draws. A low MMD² with a broad '
    'kernel need not imply legible digits: global similarity can overlook image '
    'structure. This is why visual samples, bandwidth sensitivity and classifier '
    'diagnostics are all reported. Gaussian-family samples can exceed the valid '
    'pixel range; raw-sample MMD and the out-of-range table expose this limitation.',
    'Class proportions are reported both for all predictions and conditionally on '
    'confidence >= 0.6. Total variation (TV) from uniform is half the sum of absolute '
    'deviations from 0.1; zero is exactly balanced. Confidence filtering is only a '
    'recognizability proxy: forests can be confidently wrong off the data manifold, '
    'and filtering may favor some classes. Threshold sensitivity is therefore '
    'reported, and no claim of true semantic balance is made from RF predictions alone.']
for row in digit_test_scores.itertuples():

    lines.append(f'**{row.model}:** {row.confident_fraction:.1%} of draws pass '
                 f'the confidence threshold; TV among these is {row.TV_confident:.3f} '
                 f'(all draws: {row.TV_all:.3f}).')
lines.append('Limitations: two digit repetitions and a small validation set give '
    'no precise uncertainty interval. Hyperparameter grids and the 2-tree forest '
    'are intentionally limited for runtime. Results are descriptive, not proofs of '
    'optimality. Scaling, jitter, covariance regularization and clipping conventions '
    'are explicit so comparisons can be reproduced.')
central_means = digit_results.query("kernel == 'se' and factor == 1 and n == 1200")
kde_means = central_means[central_means.family == 'KDE'].groupby('param').mmd.mean()
lines.insert(2, f'At n=1,200, validation SE MMD² for KDE is '
    f'{kde_means.loc[.05]:.5f} at bandwidth 0.05 and '
    f'{kde_means.loc[.4]:.5f} at bandwidth 0.4. The broad bandwidth is clearly '
    'worse on this metric. In contrast, several flexible-model differences '
    'are similar in size to the between-run variation; they do not establish '
    'a reliable fine-grained ranking. More training data does not improve '
    'every observed estimate monotonically, as expected for finite-sample MMD.')
display(Markdown('\n\n'.join(lines)))
```

Digit observations

The independent-test SE ranking is: **KDE → GMM → Forest → Gaussian** (lower MMD² first). The table reports IMQ separately; agreement between kernels is stronger evidence than one score alone.

At n=1,200, validation SE MMD² for KDE is -0.00076 at bandwidth 0.05 and 0.16595 at bandwidth 0.4. The broad bandwidth is clearly worse on this metric. In contrast, several flexible-model differences are similar in size to the between-run variation; they do not establish a reliable fine-grained ranking. More training data does not improve every observed estimate monotonically, as expected for finite-sample MMD.

A full single Gaussian models global covariance but averages distinct digit modes. A mixture separates some modes; the diagonal restriction loses within-component pixel correlations, and more components use more parameters. KDE preserves local training images at a small bandwidth, but this does not by itself establish novel sample generation. Large KDE bandwidths inject visible pixel noise. The supplied forest partitions the data and fits local full Gaussians; its minimum leaf size trades local specialization against covariance stability.

The displayed samples are unfiltered random draws. A low MMD² with a broad kernel need not imply legible digits: global similarity can overlook image structure. This is why visual samples, bandwidth sensitivity and classifier diagnostics are all reported. Gaussian-family samples can exceed the valid pixel range; raw-sample MMD and the out-of-range table expose this limitation.

Class proportions are reported both for all predictions and conditionally on confidence >= 0.6. Total variation (TV) from uniform is half the sum of absolute deviations from 0.1; zero is exactly balanced. Confidence filtering is only

a recognizability proxy: forests can be confidently wrong off the data manifold, and filtering may favor some classes. Threshold sensitivity is therefore reported, and no claim of true semantic balance is made from RF predictions alone.

Gaussian: 3.6% of draws pass the confidence threshold; TV among these is 0.294 (all draws: 0.162).

GMM: 69.9% of draws pass the confidence threshold; TV among these is 0.101 (all draws: 0.028).

KDE: 87.3% of draws pass the confidence threshold; TV among these is 0.043 (all draws: 0.027).

Forest: 16.0% of draws pass the confidence threshold; TV among these is 0.244 (all draws: 0.122).

Limitations: two digit repetitions and a small validation set give no precise uncertainty interval. Hyperparameter grids and the 2-tree forest are intentionally limited for runtime. Results are descriptive, not proofs of optimality. Scaling, jitter, covariance regularization and clipping conventions are explicit so comparisons can be reproduced.

References and submission contents

• Assignment: `ex01.pdf`, Exercise 1, Winter Semester 2026/27, U. Köthe.

• Supplied forest: https://tinyurl.com/HD-GNN-density-forest-py (resolved to https://heibox.uni-heidelberg.de/f/9bd2fca2e0174ea48f87/).

• Forest background cited in the supplied source: Criminisi, Shotton & Konukoglu (2011), *Decision Forests*, Chapter 5.

• sklearn is used only for data generation/loading in Part 1; all four Part 1 density estimators and MMD are implemented above. Part 2 uses sklearn as requested.

The submission archive contains this executed notebook, its PDF export, the supplied forest, requirements, and CSV experiment results. To reproduce, install requirements and run all notebook cells from the directory containing `density_forest.py`.